

Floating Point Arithmetic

A number is *floating point number* if it contains a `.`

In [125…
```
12.3
```
Out[125… 12.3

In [126…
```
.1
```
Out[126… 0.1

You can also use *scientific notation* when writing floating point numbers:

In [127…
```
1.2354e3
```
Out[127… 1235.4

An integer can be written as a floating point number by adding `.` after it.

In [128…
```
12.
```
Out[128… 12.0

Use `+` for addition, `-` for subtraction, `*` for multiplication and `**` for exponentation like integer arithmetic.

In [129…
```
6.4*(1.25-12.)**5.
```
Out[129… -918802.76875

Also you can use `//` and `%`

In [130…
```
2.8 // 1.2
```
Out[130… 2.0

In [131…
```
2.8 % 1.2
```
Out[131… 0.3999999999999999

For (true) division use: `/`

In [132…
```
15.3 / 4.2
```
Out[132… 3.642857142857143

The (true) division is always done using floating point arithmetic. Even if the result is integer.

In [133…
```
12 / 3
```
Out[133… 4.0

When doing arithmetic if any of the operands is a floating point number then *floating point*

arithmetic is performed.

In [134…
```
12 + 13.
```

Out[134…
```
25.0
```

Floating point arithmetic is very similar to integer arithmetic but uses different algorithms for computation.

Warning: Floating point arithmetic sometimes gives surprising results. For example:

In [135…
```
0.1 * 3
```

Out[135…
```
0.30000000000000004
```

Therefore when working on floating point numbers you should know that the result may not be exact.

Unlike integers floating point numbers can **not** be arbitrary big.

In [158…
```
2.0**1023
```

Out[158…
```
8.98846567431158e+307
```

To get some floating point number `number` rounded to `n` digits use `round(number,ndigits=n)`

In [137…
```
round(12.34567, ndigits=3)
```

Out[137…
```
12.346
```

`math` Module

Modules are containers for predefined functions other then the builtin ones. To use more mathematical functions use `math` module. One way of using `math` module is:

In [138…
```
from math import *
```

Then you can use many mathematical functions defined there.

In [139…
```
sin(2.3)
```

Out[139…
```
0.7457052121767203
```

In [140…
```
log10(123)
```

Out[140…
```
2.089905111439398
```

Complex Number Arithmetic

Python also supports complex number arithmetic. Any complex number of the form $a + bi \in C$ can be represented by `a+bj` in Python.

In [141…
```
((3 + 1j) * 2) ** 2
```

Out[141… (32+24j)

To get the real and imaginary part of a number use `.real` and `.imag` properties.

In [142…
```
(2 + 3j).real
```

Out[142… 2.0

In [143…
```
(2 + 3j).imag
```

Out[143… 3.0

To get the absolute value of a complex number use `abs()` function.

In [144…
```
abs(3+4j)
```

Out[144… 5.0

`cmath` Module

To use the mathematical functions defined on complex numbers use `cmath` istead of `math` module. One way of using `cmath` module is:

In [145…
```
from cmath import *
```

Then you can use mathematical functions similar to `math` module.

In [146…
```
sin(2.3-1j)
```

Out[146… (1.1506832721906894+0.7830083755042921j)

In [147…
```
log10(123-3j)
```

Out[147… (2.090034250450609-0.010590448642664601j)